

Rapport final RLMinimax

14/05/2026

Projet Pac-Man - Algorithmique 4 - Fonction affine, poids appris et resultats finaux

A4 portrait

10/10

layouts meilleurs

+325.3

gain moyen

7

features / poids

0.0

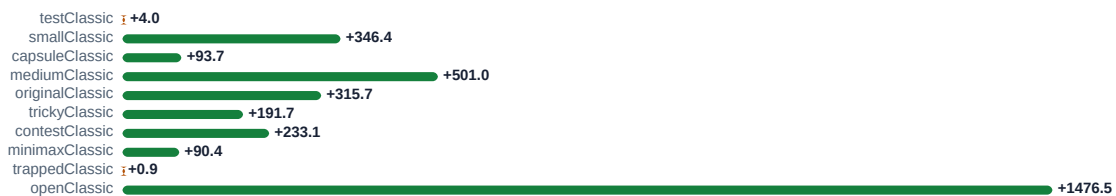
Stop moyen

Conformite au sujet

Exigence	Preuve dans le projet	OK
Facteurs Pac-Man	features.py : nourriture, fantomes, capsules, score, nourriture restante	OK
Recherche	BFS pour les distances reelles dans le labyrinthe	OK
Fonction affine	rl_evaluation_function = sum(ai * xi) + C	OK
Alpha-Beta	RLMinimaxAgent : Pac-Man MAX, fantomes MIN, coupure alpha >= beta	OK
Poids appris	train.py ajuste les coefficients par mise a jour TD	OK
Comparaison	compare.py : score moyen, winrate, temps, coups, Stop	OK

Synthese : le correctif est termine pour la partie technique. Le seul point hors code a confirmer est le depot Git/Gitesi.

Gain de score moyen RLMinimax vs Minimax



Fonction affine et poids actuels

Modele impose par le sujet : $f(s) = a_1 \cdot x_1(s) + a_2 \cdot x_2(s) + \dots + a_7 \cdot x_7(s) + C$. Verification automatique : 7 poids pour 7 features, et la valeur codee correspond a $\sum(a_i \cdot x_i) + C$.

Facteur	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Chercher nourriture
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Eviter danger
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Profiter opportunités
x4	remaining_food	0.4036	Nourriture restante	Finir la carte
x5	nearest_capsule_bfs	0.0252	Capsule proche	Effet positif faible
x6	current_score	1.8572	Score normalise	Score important
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Rester loin
C	bias	-0.0834	Correction globale	Constante

Resultats finaux relances

Layout	D	N	Minimax	AlphaBeta	RLMinimax	Diff RL	Win RL
testClassic	1	200	548.5	548.5	552.5	+4.0	100%
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	11%
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	90%

Lecture : Diff RL est positive partout. RLMinimax fait donc mieux que Minimax et AlphaBeta en score moyen sur tous les layouts testes.

Details RLMinimax

Layout	RL vs AlphaBeta	Win RL	Temps RL	Stop RL
testClassic	+4.0	100%	0.016s	0.0
smallClassic	+346.4	34%	0.284s	0.0
capsuleClassic	+93.7	12%	0.304s	0.0
mediumClassic	+501.0	28%	0.219s	0.0
originalClassic	+315.7	2%	1.430s	0.0
trickyClassic	+191.7	2%	0.643s	0.0
contestClassic	+233.1	28%	0.214s	0.0
minimaxClassic	+90.4	34%	0.036s	0.0
trappedClassic	+0.9	11%	0.001s	0.0
openClassic	+1802.4	90%	0.521s	0.0

Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur. Sur openClassic, les baselines dépassent le timeout choisi : c est une limite honnete a expliquer a l oral.

Justification algorithmique

Question	Reponse simple
Pourquoi BFS ?	Distance reelle dans le labyrinthe, plus pertinente que Manhattan avec des murs.
Pourquoi Alpha-Beta ?	Meme logique que Minimax, mais moins de branches explorees.
Pourquoi affine ?	Le sujet impose une somme ponderee des facteurs, sans termes croises.
Pourquoi apprendre ?	Les poids viennent d un apprentissage, pas d un choix manuel arbitraire.
Pourquoi comparer ?	Prouver que les poids appris ameliorent le comportement.

Phrase a dire a l oral : notre agent utilise Alpha-Beta pour explorer les coups possibles ; quand il evalue une position, il utilise une fonction affine dont les facteurs viennent du jeu et dont les poids ont ete appris par renforcement.

Commandes reproductibles

But	Commande
Syntaxe	python -m py_compile compare.py game.py pacman.py layout.py train.py rlMinimaxAgents.py features.py multiAgents.py ghostAgents.py textDisplay.py util.py
Comparer	python compare.py --num-games 100 --layout smallClassic --depth 2 --baseline both
Lancer RL	python pacman.py -p RLMinimaxAgent -l testClassic -a depth=1 -q -n 1 -f

Conclusion finale
Le projet respecte le sujet : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris, resultats statistiques et justification claire. Le rapport est pret pour la partie technique ; verifier seulement le depot Git/Gitesi avant remise.